44 HOW THE `students` TABLE WAS CREATED — synchronize, ENTITY → TABLE, MIGRATIONS

Mongo creates a collection the first time you insert into it — no setup. Postgres is different: a table must EXIST, with every column and type defined, BEFORE the first INSERT. Yet in this project nobody ran CREATE TABLE. This file explains exactly how the table got there, what it looks like for real, and the three ways tables can be created.

SECTION 1 — THE ONE SETTING THAT DID IT

```
// src/app.module.ts
TypeOrmModule.forRootAsync({
  useFactory: (configService: ConfigService) => ({
    type: 'postgres',
    url: configService.getOrThrow<string>('DATABASE_URL'),
    autoLoadEntities: true,   // <- tells TypeORM WHICH entities exist
    synchronize: true,        // <- tells TypeORM to MAKE THE DB MATCH them
  }),
}),
```

autoLoadEntities	collects every entity passed to forFeature([...]) → Student (from student-pg.module.ts)
synchronize	on every app start, compare entities with the database and run CREATE / ALTER to make them equal

SECTION 2 — STEP BY STEP, WHAT HAPPENED ON THE FIRST `npm run start`

1. ConfigModule loads .env → DATABASE_URL available
2. TypeOrmModule connects to Supabase (pg driver, SSL)
3. autoLoadEntities: StudentPgModule's forFeature([Student]) adds Student to the list of known entities
4. TypeORM reads entity METADATA from student.entity.ts: table name, every property, its type, its decorator options
5. TypeORM reads the REAL database structure: SELECT ... FROM information_schema.tables / columns / constraints → no table called "students"
6. Compare (a "schema diff"): entity wants students, DB has nothing
7. Generate DDL and run it: CREATE TABLE "students" (...) ALTER TABLE "students" ADD CONSTRAINT ... (PK, UNIQUE)
8. Continue booting: repositories ready, routes mapped, listen(3000)

Second start onwards: steps 5-6 find the table already matches → nothing is executed.

SECTION 3 — ENTITY → TABLE, DECORATOR BY DECORATOR

```
@Entity('students')                    -> table "students"
export class Student {
```

```
    @PrimaryGeneratedColumn()              -> "id" SERIAL + PRIMARY KEY
    id!: number;

    @Column()                              -> "name" character varying NOT NULL
    name!: string;

    @Column()                              -> "age" integer NOT NULL
    age!: number;

    @Column({ unique: true })              -> "email" character varying NOT NULL + UNIQUE
    email!: string;

    @Column({ nullable: true })            -> "course" character varying NULL
    course?: string;

    @CreateDateColumn()                    -> "createdAt" timestamp NOT NULL DEFAULT now()
    createdAt!: Date;

    @UpdateDateColumn()                    -> "updatedAt" timestamp NOT NULL DEFAULT now()
    updatedAt!: Date;                         (TypeORM also sets it on every UPDATE)
  }

How the TYPE is picked: TypeScript type -> Postgres type
  string   -> character varying (varchar)
  number   -> integer
  boolean  -> boolean
  Date     -> timestamp without time zone
(TypeScript emits this type info because tsconfig has
 emitDecoratorMetadata: true — that's how TypeORM "sees" the types.)
```

How NULL is picked:

```
@Column() is NOT NULL by default. `?` in TypeScript does NOT make a
column nullable — only { nullable: true } does.
```

SECTION 4 — THE SQL IT GENERATED

```
CREATE TABLE "students" (
  "id"        SERIAL NOT NULL,
  "name"      character varying NOT NULL,
  "age"       integer NOT NULL,
  "email"     character varying NOT NULL,
  "course"    character varying,
  "createdAt" TIMESTAMP NOT NULL DEFAULT now(),
  "updatedAt" TIMESTAMP NOT NULL DEFAULT now(),
  CONSTRAINT "UQ_25985d58c714a4a427ced57507b" UNIQUE ("email"),
  CONSTRAINT "PK_7d7f07271ad4ce999880713f05e" PRIMARY KEY ("id")
);
```

SERIAL is shorthand. Postgres expands it into:

```
  CREATE SEQUENCE students_id_seq;                -- a counter object
  "id" integer NOT NULL DEFAULT nextval('students_id_seq')
Every INSERT without an id asks the sequence for the next number.
Sequences never go backwards — that's why after deleting id 1 the
next student got id 3, not 1.
```

Constraint names (PK_7d7f..., UQ_2598...) are hashes TypeORM makes from the table + column names, so they're unique and stable.

```
Double quotes ("createdAt"): Postgres lower-cases unquoted names, so
camelCase column names must always be quoted in raw SQL:
  SELECT "createdAt" FROM students;   -- works
  SELECT createdAt FROM students;     -- ERROR: column "createdat" does not exist
```

SECTION 5 — THE REAL TABLE (read back from Supabase)

Query used:

```
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'students' ORDER BY ordinal_position;
```

column_name	data_type	is_nullable	column_default
id	integer	NO	nextval('students_id_seq'::regclass)
name	character varying	NO	null
age	integer	NO	null
email	character varying	NO	null
course	character varying	YES	null
createdAt	timestamp without time zone	NO	now()
updatedAt	timestamp without time zone	NO	now()

Constraints:

```
SELECT conname, pg_get_constraintdef(oid)
FROM pg_constraint WHERE conrelid = 'students'::regclass;
```

PK_7d7f07271ad4ce999880713f05e	PRIMARY KEY (id)
UQ_25985d58c714a4a427ced57507b	UNIQUE (email)

Other ways to look at it:

```
Supabase -> Table Editor -> students -> "Definition" tab
psql:  \d students
pgAdmin / DBeaver: right-click table -> Generate SQL -> DDL
```

SECTION 6 — WHAT synchronize DOES WHEN THE ENTITY CHANGES

```
Entity change                       SQL synchronize runs
add  @Column() phone!: string       ALTER TABLE "students" ADD "phone" varchar NOT NULL
```

```
                                    (FAILS if rows exist — they'd have NULL phone;
                                     use nullable: true or a default)
add  @Column({ default: 'active' })  ADD "status" varchar NOT NULL DEFAULT 'active'
change type  age: number -> string   DROP COLUMN "age"; ADD "age" varchar   <- DATA LOST
rename  course -> subject            DROP COLUMN "course"; ADD "subject"    <- DATA LOST
remove a property                    DROP COLUMN                             <- DATA LOST
add unique: true                     ADD CONSTRAINT ... UNIQUE (fails on duplicates)
delete the entity                    nothing — table is left alone

synchronize can't tell a RENAME from "drop one, add another", so it
drops. Fine in dev with test data; a disaster in production.
-> synchronize: true is DEV ONLY.
```

SECTION 7 — THE THREE WAYS TO CREATE TABLES

```
WAY 1 — synchronize: true  (what this project does)
  + zero SQL, instant, perfect while learning / prototyping
  - can drop data, no history, no review, runs on every boot
```

WAY 2 — MIGRATIONS (the production way) Turn off synchronize, keep the SQL in versioned files:

```
a) data-source.ts (the CLI needs its own config, outside Nest):
     import 'dotenv/config';
     import { DataSource } from 'typeorm';
     export default new DataSource({
       type: 'postgres',
       url: process.env.DATABASE_URL,
       ssl: { rejectUnauthorized: false },
       entities: ['src/**/*.entity.ts'],
       migrations: ['src/migrations/*.ts'],
     });
```

b) generate — diff entities vs DB, write the SQL to a file:

```
npx typeorm-ts-node-commonjs migration:generate src/migrations/CreateStudents -d data-source.ts
```

creates src/migrations/1760000000000-CreateStudents.ts:

```
export class CreateStudents1760000000000 implements MigrationInterface {
  async up(q: QueryRunner) {           // apply
    await q.query(`CREATE TABLE "students" (...)`);
  }
  async down(q: QueryRunner) {         // undo
    await q.query(`DROP TABLE "students"`);
  }
}
```

c) run / undo:

```
      npx typeorm-ts-node-commonjs migration:run    -d data-source.ts
      npx typeorm-ts-node-commonjs migration:revert -d data-source.ts

  d) in app.module.ts:  synchronize: false
     (optionally migrationsRun: true to apply pending ones at boot)
```

```
  TypeORM records applied migrations in a `migrations` table, so each
  one runs exactly once per database.
  + reviewed, in git, reversible, same steps on every environment
  + a rename can be written by hand as ALTER TABLE ... RENAME COLUMN
    (keeps the data)
  - a few more commands

WAY 3 — BY HAND (raw SQL)
  Paste the CREATE TABLE from Section 4 into Supabase -> SQL Editor
  (or psql / pgAdmin), keep synchronize: false, and make sure the
  entity matches the table exactly.
  + full control, best for learning SQL (theory/37)
  - entity and table can drift apart; nothing keeps them in sync
```

Comparison:

```
                synchronize   migrations    by hand
writes SQL      TypeORM       TypeORM (+you) you
when it runs    every boot    when you run  when you run
history / undo  none          yes (up/down) none
data safety     risky         safe          depends on you
use for         learning/dev  production    learning SQL
```

SECTION 8 — MONGO VS POSTGRES ON "CREATING THE TABLE"

```
Mongo (student.schema.ts):
  @Schema() only validates in the APP. The collection appears on the
  first insert; documents can have any shape if inserted another way.
Postgres (student.entity.ts):
  The entity is turned into a real TABLE with types and constraints
  enforced by the DATABASE. A row with a missing name or duplicate
  email is rejected by Postgres itself, no matter who inserts it.
```

★ RULE OF THUMB

The table exists because synchronize: true compared student.entity.ts with the database on boot and ran CREATE TABLE for you: each decorator became a column, each option a constraint, SERIAL became a sequence. Great for learning — but it drops data on renames, so production switches to migrations (generated SQL files you review and run), or you write the CREATE TABLE yourself.

See also: **theory/43-where-sql-queries-come-from.txt** (the DML side), **theory/41-supabase-postgres-crud-full-steps.txt** (the full build), **theory/40-postgres-end-to-end.txt** (STEP 12 migrations), **theory/37-sql-cheatsheet.txt** (CREATE / ALTER TABLE syntax)